

RideFlow Development Styles and Engineering Patterns Guide

Project: RideFlow

Repository: <https://github.com/raiven427/ride-flow>

Audience: The owner and any developer maintaining or extending RideFlow

Purpose: Explain the development styles, architectural patterns, visual conventions, and engineering practices used in the project, including when each style should be used.

A development style is not only the appearance of a page. It is the repeated way a team designs screens, structures code, validates data, protects users, tests behavior, deploys releases, and responds to failure.

1. Product and visual development style

Editorial Transit design

RideFlow uses an **Editorial Transit** visual style. It combines warm sand surfaces, deep ink/navy text, restrained coral-orange actions, large editorial typography, asymmetrical panels, soft rounded cards, and transport-oriented imagery. It is intended to feel calm, premium, legible, and more human than a generic dashboard.

Pattern	How it appears in RideFlow	When to use it
Warm neutral background	Sand-colored workspace surfaces	Main app backgrounds and quiet areas
Ink/navy hierarchy	Dark headings and navigation text	Primary reading hierarchy and contrast
Coral action color	Booking and primary action buttons	One high-priority action per region
Asymmetrical layouts	Split hero panels and mixed-width cards	Marketing and overview moments
Editorial typography	Large display headings with compact supporting copy	Hero, empty states, section intros
Soft corners	Rounded cards and controls	Comfortable, friendly product surfaces
Restrained motion	Short transitions, spring-like feedback, reduced-motion support	Confirmation, drawers, hover, selection

Do not turn every screen into a colorful card grid. Use contrast and whitespace to create hierarchy. A dashboard should explain the next useful action rather than display decoration.

Apple-inspired interaction style

The app uses small, responsive interactions instead of long cinematic animations. Buttons confirm a press quickly. Cards transition with opacity and transform. Drawers and modals enter from an intentional origin. Grouped content can reveal with a small stagger. Non-essential motion must respect `prefers-reduced-motion`.

Use transitions primarily on `transform` and `opacity`. Avoid animating layout-heavy properties such as width, height, margin, or top/left unless the behavior truly requires it. Keep ordinary interaction feedback near 100–300 milliseconds.

Accessibility style

Every interaction should be keyboard reachable, have a visible focus state, use readable contrast, and expose a meaningful label to assistive technology. Empty states should explain what happens next. Error messages should describe recovery rather than only state that something failed.

2. Frontend architecture style

RideFlow uses a **component-based React style**. The interface is composed from page-level experiences, reusable layout components, reusable UI primitives, and small stateful sections.

Layer	Example	Responsibility
Application shell	<code>client/src/App.tsx</code>	Routes, providers, theme, error boundary
Page experience	<code>client/src/pages/Home.tsx</code>	Customer, driver, profile, booking, admin workflows
Shared layout	<code>DashboardLayout.tsx</code>	Consistent internal navigation and structure
UI primitive	<code>client/src/components/ui/button.tsx</code>	Reusable accessible control
Client data layer	<code>client/src/lib/trpc.ts</code>	Typed server communication
Auth state	<code>useAuth.ts</code>	Current user and login/logout behavior
Global styling	<code>client/src/index.css</code>	Tokens, responsive rules, motion, visual system

Component composition

Prefer composing smaller components over copying markup. A component should have one clear responsibility. If a page section needs its own state, loading behavior, or tests, extract it when that improves clarity.

Use props to pass data and callbacks. Keep provider calls, database logic, and secrets out of browser components. The browser may request an operation; it should not become the authority for permissions or financial totals.

State-management style

Use local React state for temporary UI state such as selected tabs, open drawers, and form fields. Use tRPC/React Query for server state. Keep query inputs stable with `useMemo` or state

when an object or array is used as an input. Show loading, success, empty, error, and retry states.

Do not put every value in global state. Global state should be reserved for information genuinely shared across unrelated areas, such as authentication or theme.

Form style

Forms should validate obvious problems near the input, disable submission while an operation is in flight, show server errors, and provide a success confirmation. Sensitive forms should not echo secrets back to the browser or logs.

3. API and backend development style

RideFlow uses a **typed procedure-first API style** with tRPC. Instead of writing a separate REST controller and manually duplicating types in the frontend, the server exposes procedures that the typed client consumes.

Procedure categories

Procedure type	Intended access
Public	No signed-in user required
Protected	Any authenticated user
Admin	Authenticated user with administrator role

Authorization belongs in server middleware and procedures. Conditional navigation in React improves usability but is not a security boundary.

Procedure structure

A well-structured procedure should:

1. Receive a typed input.
2. Confirm the authentication requirement.
3. Confirm role or ownership authorization.
4. Validate business rules.
5. Call a database, storage, or provider helper.
6. Record a meaningful operational event when appropriate.

7. Return a typed result or a useful typed error.

Keep domain logic in helpers such as `server/db.ts`, `server/fare.ts`, `server/storage.ts`, or `server/payments.ts` instead of putting every detail inside one router file.

Error-handling style

Errors should tell the user or developer what can be done next without exposing secrets or internal infrastructure. Use a forbidden error for authorization failures, validation errors for malformed input, and internal/provider errors for failures that need logging and retry handling.

Do not catch an error and silently return success. Do not expose stack traces, database credentials, access tokens, or payment-provider secrets in responses.

4. Domain-driven feature style

RideFlow organizes behavior around domain concepts: users, profiles, files, fare rules, fare quotes, ledger entries, presence, activity, admin settings, and payments. When adding a feature, use the vocabulary of the domain rather than generic names such as `data`, `item`, or `thing`.

Feature development sequence

Requirement

- domain model and ownership decision
- schema/migration if durable data is needed
- database helper
- protected/public/admin procedure
- frontend state and UI
- tests
- browser verification
- documentation and deployment notes

This is a **vertical-slice style**: a feature is complete only when its data model, server contract, UI behavior, tests, and operating instructions agree.

5. Database development style

RideFlow uses **schema-first relational development** with Drizzle and MySQL/TiDB. The source of truth is `drizzle/schema.ts`; migration files record how a database reaches the

required structure.

Migration style

```
# Update the source schema
pnpm drizzle-kit generate

# Inspect the generated SQL
# Test against a disposable database
pnpm drizzle-kit migrate
```

Never change production tables casually through a dashboard. Review generated SQL, back up production, consider existing data, and test both the forward migration and application behavior afterward.

Financial and audit style

Fare quotes and ledger entries represent business history. The server calculates the 5% commission using basis points. Prefer append-only financial entries and explicit refund/reversal records rather than silently rewriting historical values.

Activity events are append-only operational records. They should contain a safe summary and minimal metadata. Do not put passwords, payment credentials, raw files, or full GPS streams into the activity feed.

Ownership and authorization style

Every sensitive row should have a clear owner or access rule. A developer must be able to answer who may read, create, update, or delete it. Admin settings and driver documents require stronger access than a public fare rule.

6. Storage development style

RideFlow uses a **metadata-in-database, bytes-in-object-storage** pattern. The database stores the file owner, purpose, MIME type, size, review status, storage key, and provider URL. S3-compatible storage stores the actual bytes.

Keep buckets private. Generate short-lived signed URLs for authorized access. Validate file type and size on the server. Never rely on the web host's local filesystem for durable uploads because free and autoscaling services may discard it on restart.

When adding a new file purpose, update the schema enum, validation, authorization, UI copy, retention policy, and tests together.

7. Authentication and authorization style

RideFlow uses provider-neutral OIDC/OAuth configuration. The identity provider owns passwords and account authentication. RideFlow maps the authenticated identity to a local user and role.

Use a **defense-in-depth style**:

Layer	Protection
Identity provider	Password, MFA, account recovery
Session	Secure cookie and server context
Procedure	Protected/admin middleware
Domain rule	Ownership and business validation
Storage	Private bucket and signed URLs
Audit	Sign-in and admin activity events
Operations	Secret manager, rotation, logs, alerts

Never trust a client-provided user ID, role, payment status, or price. Derive identity from the authenticated server context and calculate sensitive totals on the server.

8. Payment and webhook development style

Payment integrations use an **asynchronous, provider-verified, idempotent style**. A browser request is not proof of a completed payment. A provider webhook or callback must be verified, matched to a pending transaction, and handled safely if delivered more than once.

Stripe style

Use test mode during development. Verify the raw request body and `Stripe-Signature` header. Record provider event IDs and make repeated events harmless. Test success, decline, invalid signature, duplicate event, timeout, refund, and dispute scenarios.

Daraja style

M-Pesa STK Push is asynchronous. The initial request response means the provider accepted the request; the callback determines the final result. Store provider request identifiers as pending, match the callback, then resolve the transaction. Never treat a browser message as final payment confirmation.

Commission style

The 5% platform commission is represented as 500 basis points. Calculate rider charge, platform commission, and driver earnings on the server. Record ledger entries consistently and reconcile them with the provider transaction before payout.

9. Realtime and dispatch development style

Realtime GPS and dispatch should use a **separate authoritative trip state plus realtime transport** pattern. WebSockets or a managed realtime service may deliver updates, but the main server/database remains authoritative for trip assignment and payment state.

A secure location design should authenticate the connection, authorize the user to a particular trip channel, limit update frequency, handle reconnects, expire access when a trip ends, and avoid publishing a driver's exact location to unauthorized users. Store the latest operational location separately from the general activity feed.

Dispatch should be idempotent. If two workers or two requests try to assign a driver, the database and server rules must prevent conflicting assignments.

10. Testing style

RideFlow uses **regression-focused automated testing** with Vitest. Existing tests cover auth logout, admin authorization, sign-in activity, fare calculations, operations access, payments, storage, and success paths.

Test layers

Layer	What it proves
Unit test	A calculation/helper behaves correctly in isolation
Procedure test	A tRPC action authorizes and returns the correct result
Integration test	Database/storage/provider boundaries work together
Browser test	A real user can complete a visible flow
Production smoke test	Deployed health, auth, callbacks, and logs work

For every new behavior, include the success path and at least one failure or unauthorized path. Test duplicate payment callbacks and stale/replayed requests when the feature is financial or operational.

Run:

```
pnpm check
pnpm test
pnpm build
```

11. Git and release style

RideFlow uses **small, reviewable, reversible Git changes**. Before editing, add the requirement to `todo.md`. After completing it, mark the item done, review `git diff`, run checks, save a checkpoint, and push the approved source to the private repository.

Useful commands:

```
git status --short
git diff
git log --oneline --decorate -10
git switch -c feature/name
git add <reviewed-files>
git commit -m "Describe the change"
git push -u origin feature/name
```

Do not force-push main casually. Do not store secrets in Git. A code checkpoint is not a database backup; maintain both recovery plans.

12. Deployment and configuration style

Use an **environment-driven, provider-neutral deployment style**. Code should not contain host-specific secrets, hardcoded database URLs, fixed production ports, or assumptions that the local filesystem is durable.

Configuration	Style
Secrets	Host secret manager, never committed files
Database	<code>DATABASE_URL</code> , TLS, least-privilege user
Port	Provider-supplied <code>PORT</code>
Host binding	Provider-compatible public binding
Files	Private S3-compatible storage
Auth	Exact HTTPS callback and secure cookies
Payments	Sandbox first, signed callbacks, idempotency
Release	Build, test, deploy, health check, smoke test
Recovery	Checkpoint/rollback plus database and storage backups

The standard developer sequence is `pnpm install --frozen-lockfile` , `pnpm check` , `pnpm test` , `pnpm build` , deploy, inspect logs, and run browser smoke tests.

13. Operations and observability style

RideFlow uses **meaningful activity logging** rather than logging every implementation detail. Sign-in, quote creation, uploads, notification changes, and admin ownership changes can be visible to the admin operations feed. Presence heartbeats show recent online status.

Logs should help answer what happened, when it happened, which non-sensitive user or transaction ID was involved, and whether the operation succeeded. Do not log secrets, passwords, full payment details, raw identity documents, or unnecessary location history.

Define retention for activity and location data. Monitor health checks, database errors, storage failures, webhook failures, authentication loops, stale heartbeats, and backup jobs.

14. Documentation and handoff style

The project uses **operational documentation as part of development**. A feature is not finished if only the code changed. The developer should update the relevant README or guide with setup variables, migration steps, test commands, callback URLs, troubleshooting, and rollback notes.

Good documentation names exact files, commands, assumptions, risks, and unfinished work. It does not claim that a scaffold is production-ready when real provider configuration or operational controls are still missing.

15. When to use each style

Situation	Preferred style
New customer-facing screen	Editorial Transit visual system, component composition, accessible states
New server action	Typed tRPC procedure with protected/admin authorization
New durable business concept	Schema-first vertical slice with migration, helper, procedure, UI, and tests
File upload	Private object storage with database metadata and signed URLs
Payment action	Provider-verified asynchronous callback with idempotent ledger behavior
GPS/dispatch	Authenticated realtime transport with authoritative trip state
Security-sensitive change	Defense-in-depth server checks, secret rotation, regression tests
Bug fix	Reproduce, isolate, patch minimally, add regression test, document
Release	Small Git change, full validation, checkpoint, deploy, smoke test
Handoff	Explicit code map, provider requirements, unfinished work, recovery plan

16. Anti-patterns to avoid

Do not hardcode demo earnings, fake reviews, fake ratings, or fake operational data. Do not trust browser-calculated fares or payment success. Do not put file bytes in database columns. Do not use public URLs for private driver documents. Do not hide an admin action and assume it is protected. Do not edit generated migrations without understanding the schema. Do not use local disk for durable uploads. Do not add a new provider without documenting secrets, callbacks, failure modes, and tests.

Do not create a large feature by changing the UI first and postponing the domain model. Do not swallow errors. Do not make every feature global state. Do not mix unrelated refactors with a

payment or schema change. Do not delete old documentation that explains operational history.

17. Practical review checklist

Before approving a change, ask the following questions:

1. Does the change follow the existing visual and component style?
2. Is the server responsible for authorization and sensitive calculations?
3. Is the data model clear and migrated safely?
4. Are loading, empty, success, error, retry, and unauthorized states present?
5. Are tests included for success, failure, and access control?
6. Are secrets and private files protected?
7. Are provider callbacks signed, asynchronous, and idempotent?
8. Is activity logging useful without collecting unnecessary sensitive data?
9. Can the change be deployed, monitored, backed up, and rolled back?
10. Has the documentation been updated for the next developer?

References

- [1]: <https://react.dev/learn> React, “Learn React.”
- [2]: <https://www.typescriptlang.org/docs/> TypeScript, “Documentation.”
- [3]: <https://trpc.io/docs> tRPC, “Documentation.”
- [4]: <https://orm.drizzle.team/docs/overview> Drizzle ORM, “Overview.”
- [5]: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API MDN Web Docs, “WebSocket API.”
- [6]: <https://docs.stripe.com/webhooks> Stripe, “Receive Stripe events in your webhook endpoint.”
- [7]: <https://owasp.org/www-project-top-ten/> OWASP, “OWASP Top 10.”
- [8]: <https://git-scm.com/book/en/v2> Git, “Pro Git.”
- [9]: <https://dev.mysql.com/doc/> MySQL, “MySQL Documentation.”